



UNIVERSITÀ DEGLI STUDI
DI SALERNO

Azure Serverless RCE

Una Piattaforma Cloud Message-Driven per la Compilazione ed Esecuzione Remota
di Codice

Corso di Cloud Computing

A.A. 2025/26

Lorenzo Sorrentino

Panoramica del Progetto

- ▷ **Compilazione Source-to-Source:** Traduzione automatica di codice custom in eseguibile C, sfruttando un transpiler Java (JFlex/CUP).
- ▷ **Esecuzione in Sandbox Sicura:** Avvia il codice non attendibile dell'utente all'interno di container Linux isolati ed effimeri.
- ▷ **Elaborazione Input Batch:** Inietta dinamicamente payload di standard input insieme all'invio del codice.
- ▷ **Cronologia Completa:** Traccia stati di esecuzione, tempi di calcolo e log tramite un database serverless NoSQL.



I 5 Microservizi Principali

Frontend

Azure App Service



Web app Python basata su Streamlit. Ospita l'interfaccia per la scrittura del codice e dell'input, mostra il risultato dell'esecuzione e la cronologia.

Messaggi e Storage

Queue & Blob Storage



Disaccoppia la UI dai processi computazionali pesanti, archiviando in sicurezza gli artefatti ed evitando timeout HTTP.

Esecuzione

Container Instances



La sandbox worker che contiene OpenJDK e le toolchain GCC. Esegue il polling della coda e processa i task in background.

Database

Cosmos DB



Datastore NoSQL serverless per la gestione della cronologia delle esecuzioni, del monitoraggio dello stato e dei log.

Il Pattern *Message-Driven*

Perché il Polling?

Posizionando Azure Queue Storage tra il frontend veloce e il motore di compilazione pesante, il sistema viene disaccoppiato. Questo garantisce la consegna dei job al 100% senza esporre la UI ai timeout delle richieste HTTP.

Il Ciclo del Worker

Un demone Python in esecuzione continua nel container preleva attivamente i task dalla coda. Questo design protegge il sistema dai sovraccarichi e impedisce a più container di collidere sull'esecuzione dello stesso codice utente.

Pipeline dei **Messaggi**

L'utente invia il codice sorgente tramite l'App Service. I file vengono salvati in modo sicuro negli Azure Blob e un leggero trigger JSON viene iniettato nella Coda.

Il worker isolato intercetta il payload, esegue il calcolo asincrono e aggiorna Cosmos DB, permettendo alla UI di notificare il completamento.



Resilienza e Gestione Timeout



Visibility Timeout (60s): Il messaggio prelevato dalla coda viene nascosto. Evita che altri worker duplicchino il job ("At-least-once delivery").

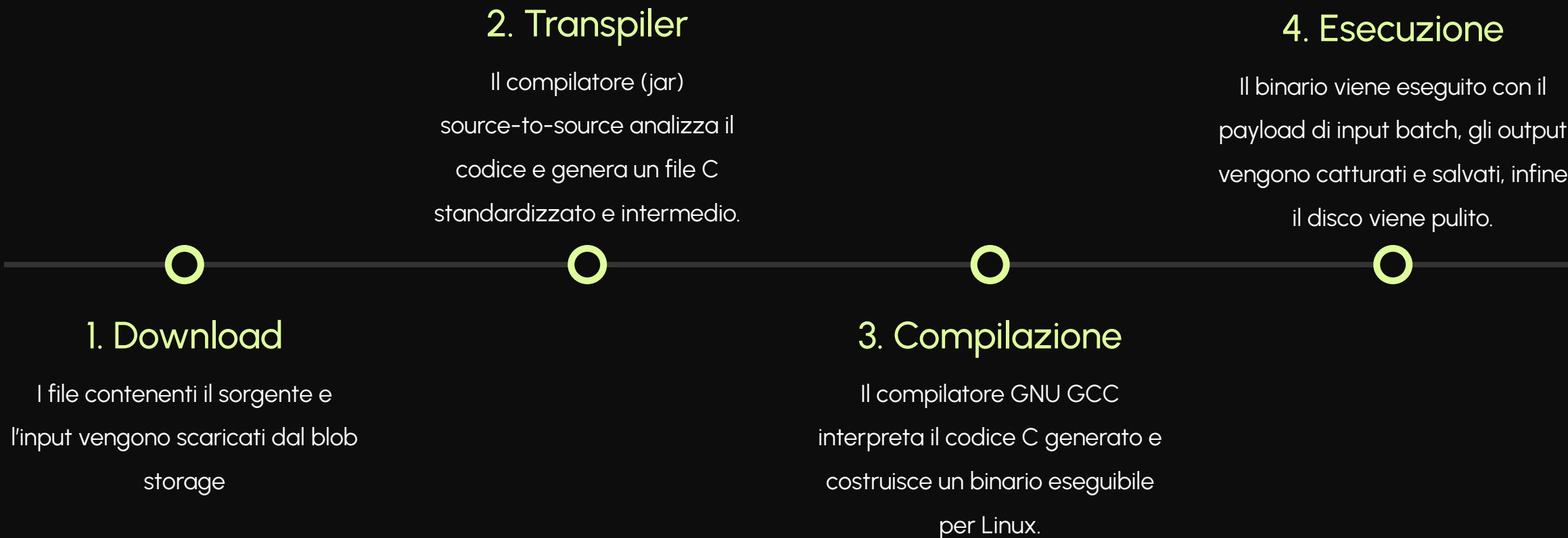


Process Timeout (40s): Python impone un hard-limit al processo in esecuzione. Se sfiorato, il processo viene ucciso dal sistema operativo prevenendo loop infiniti.



Gestione Margine: Il margine di 20s permette al worker di aggiornare Cosmos DB col log d'errore e cancellare regolarmente il messaggio dalla coda.

La Pipeline di Compilazione



Perché ACI invece di Azure Functions?

Caratteristica Architettuale	Azure Functions (Serverless)	Azure Container Instances (ACI)
Ecosistema Runtime	Singolo Linguaggio (es. solo Python)	Multi-toolchain (Python, Java, GCC)
Sicurezza e Isolamento	Infrastruttura Condivisa (Multi-tenant)	Isolamento Sandbox a livello Hypervisor
I/O su File System	Ristretto / Archiviazione di Rete	Disco Linux Effimero e Senza Restrizioni
Timeout di Esecuzione	Limite rigido a 5 - 10 Minuti	Illimitato (Controllo Python personalizzato)

Perché Queue Storage invece di Service Bus?

Caratteristica Architetturale	Azure Queue Storage	Azure Service Bus
Pattern di Messaggistica	Coda semplice (1-a-1), perfetta per il modello a code di lavoro (Task Queue)	Broker Enterprise per architetture Pub/Sub, Topic, filtri e routing complesso
Gestione Infrastruttura	Totalmente integrato nello Storage Account esistente (condiviso con i Blob)	Richiede il provisioning di un Namespace dedicato e separato
Modello di Costo	Pagamento a micro-transazioni (frazioni di centesimo), 0 costi base mensili	Costi fissi orari per i tier Standard/Premium (Over-engineering per il progetto)
Ordinamento (FIFO)	Best-effort (l'ordine cronologico rigido non è un requisito bloccante)	Garantito in modo rigoroso tramite code con sessioni (Strict FIFO)

Vantaggi Tecnici Principali



Scalabilità: Sfruttando le capacità di autoscaling di Azure App Service e la predisposizione all'aggiunta di ulteriori container backend (Es. Container App), il sistema garantisce un'ottima scalabilità.

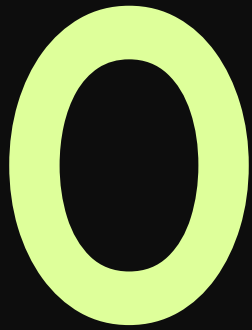


Resilienza: Il meccanismo di Visibility Timeout di Azure Queue Storage protegge l'infrastruttura dai crash, assicurando il reinserimento in coda dei job non correttamente processati.



Sicurezza: L'esecuzione del codice all'interno di container Docker isolati previene corruzioni di sistema derivanti da codice dannoso.

Automazione CI/CD



Deploy Manuali

Workflow di Continuous Delivery

Sfruttando le potenzialità di GitHub Actions, i push sul codice attivano build Docker automatiche. L'immagine aggiornata del worker viene caricata direttamente nel container registry, mentre l'App Service sincronizza in tempo reale le modifiche del frontend.

Grazie per l'attenzione

Corso di Cloud Computing | A.A. 2025/26
Lorenzo Sorrentino
